SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1741**

Register Number	**192319056**
Name	**THARIKA SHREE M**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS:2 **Total Marks:20**

Descriptive Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behaviour and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.
2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Code of Conduct:

I **Tharika Shree M (192319056)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Tharika Shree M

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Understanding of Concepts	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	5
Explanation	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	5
Application	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	4
Organization	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	3
Accuracy	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	2
Clarity	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	1

1. Intelligent Agents in a Smart Home Automation System

Introduction

An intelligent agent is a system that perceives its environment through sensors and takes actions using actuators to achieve specific objectives. In a smart home, sensors collect information such as motion, temperature, light intensity, and time of day. The system uses this information to control lighting, air conditioning, and security devices automatically.

Types of Intelligent Agents

1. Simple Reflex Agent
    a. A simple reflex agent selects actions based only on the current percept using predefined condition–action rules.
    2. It uses rules such as if motion is detected and the room is dark, then switch on the light.
    3. It responds quickly to immediate environmental changes.
    4. It is simple to design and requires less computational power.
    5. However, it does not remember previous states or learn user preferences.
        a. Example: A motion sensor automatically switches on a hallway light when someone enters.
        b. Limitation: It may switch off the light when no motion is detected, even if a person is sitting still.
2. Model-Based Agent
    a. A model-based agent maintains an internal state or model of the environment to make decisions when the current sensor information is incomplete.
    2. It combines current sensor readings with information about previous states.
    3. It can remember whether a room is occupied and track changes in environmental conditions.
    4. It makes better decisions when sensors cannot directly observe everything.
    5. However, maintaining an accurate internal model requires additional memory and computation.
        a. Example: The system remembers that a person is in the living room even when the motion sensor temporarily stops detecting movement.
        b. Limitation: Incorrect or outdated internal information may lead to inappropriate actions.
3. Goal-Based Agent
    a. A goal-based agent chooses actions by considering whether they help achieve a specified goal.
    2. It evaluates possible actions and selects those that move the system toward its target.
    3. Goals may include maintaining a comfortable temperature, reducing energy consumption, or securing the house.
    4. It can plan a sequence of actions instead of reacting only to current inputs.
    5. However, planning may require more time and computational resources.
        a. Example: If the goal is to maintain a room temperature of 24°C, the agent checks the current temperature and operates the air conditioner when necessary.
        b. Limitation: A goal-based agent may not distinguish between different solutions that achieve the same goal equally well.
4. Utility-Based Agent

    a. A utility-based agent selects actions by evaluating their expected usefulness according to a utility function. It can balance several competing objectives.
2. It considers comfort, electricity cost, energy efficiency, convenience, and security.
3. It assigns relative importance to different outcomes and chooses the action with the highest expected utility.
4. It can adapt its decisions to changing conditions and user preferences when supported by learning mechanisms.
5. However, designing a suitable utility function can be complex.
    a. Example: The system maintains a comfortable temperature while reducing air-conditioning usage when electricity prices are high, provided safety and comfort requirements are satisfied.
    b. Limitation: Poorly chosen utility weights may cause the system to prioritize energy savings over user comfort.

Comparison of Intelligent Agents

Agent type	Decision basis	Smart home application
Simple reflex	Current sensor input	Switching lights on or off
Model-based	Current input and internal state	Tracking room occupancy
Goal-based	Desired target	Maintaining a target temperature
Utility-based	Best overall expected outcome	Balancing comfort, cost, and energy use

Most Effective Approach

    a. A hybrid intelligent agent combining model-based, goal-based, and utility-based reasoning with simple reflex rules would be the most effective for a smart home.
2. Immediate response: Simple reflex rules can turn on lights when motion is detected or activate an alarm when a security threshold is crossed.
3. Environmental awareness: A model-based component tracks occupancy, temperature, and previous user activities.
4. Goal achievement: A goal-based component works toward maintaining comfortable temperatures and ensuring that doors are secured.
5. Optimization: A utility-based component balances comfort, energy consumption, and electricity costs.
6. Learning preferences: A learning component can use past user behaviour to adjust lighting brightness and temperature settings over time.
    a. Security-related actions should have strict safety rules that cannot be overridden merely to improve comfort or reduce costs.

Conclusion

A hybrid intelligent agent is the best choice because no single agent type can efficiently handle all smart home requirements. Combining immediate responses, internal state

tracking, goal-directed planning, utility optimization, and learning helps achieve comfort, energy efficiency, security, and personalization.

2. Uninformed Search Strategies for a Robot in an Unknown Maze

Introduction

An uninformed search algorithm explores possible states without using heuristic information about how close a state is to the goal. A robot placed in an unknown maze must discover possible paths, avoid obstacles, and reach the exit.

The maze can be represented as a state-space graph, where each open cell is a state and each valid movement is an edge. The initial state is the robot's starting position, and the goal state is the exit.

Two important uninformed search strategies are Uniform Cost Search (UCS) and Iterative Deepening Search (IDS).

1. Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the lowest accumulated path cost. It uses a priority queue to decide which node to explore next.

Working procedure:

1. Place the starting position in a priority queue with a path cost of zero.
2. Remove the node with the lowest accumulated cost.
3. Check whether the node is the exit. If it is, return the path.
4. Otherwise, explore its valid neighbouring cells.
5. Calculate the cost of reaching each neighbour and update the priority queue if a cheaper path is found.
6. Continue until the exit is reached or no reachable states remain.

Example: If moving through different maze cells has different costs, UCS explores the cheapest available route first rather than simply choosing the path with the fewest moves.

Advantages:

- Finds an optimal, minimum-cost path when step costs are non-negative and the required search conditions hold.
- Handles mazes in which different movements have different costs.
- Avoids committing to a costly route merely because it appears shorter in the number of steps.

Disadvantages:

- Requires considerable memory to store the frontier and explored states.
- May explore many low-cost states before reaching the goal.
- Can be computationally expensive for large mazes.

2. Iterative Deepening Search (IDS)

Iterative Deepening Search repeatedly performs depth-limited depth-first search, increasing the depth limit each time until the goal is found.

Working procedure:

1. Start with a depth limit of zero.
2. Perform depth-limited search from the starting position.
3. If the exit is not found, increase the depth limit by one.
4. Restart the search from the beginning with the new limit.
5. Continue until the exit is found or all reachable states have been exhausted.

Example: The robot first checks paths of depth zero, then depth one, then depth two, and so on, until it discovers the exit.

Advantages:

- Requires much less memory than UCS in typical tree-search implementations.
- Finds a shallowest solution when each movement has equal cost.
- Is useful when the maze's depth is unknown.

- Combines the low memory requirement of depth-first search with the completeness of breadth-first search under standard finite-branching assumptions.

Disadvantages:

- Repeats the exploration of upper-level nodes at each iteration.
- May waste time when the same states are repeatedly explored.
- Does not guarantee a minimum-cost path when movements have different costs.

Factor	Uniform Cost Search (UCS)	Iterative Deepening Search (IDS)
Search Method	Expands the node with the lowest path cost.	Repeats depth-limited DFS with increasing depth limits.
Time Complexity	Exponential; depends on path cost and minimum step cost.	\(O(b^d)\)
Space Complexity	High; stores frontier and explored nodes.	\(O(bd)\); requires less memory.
Completeness	Complete if each step cost is at least a positive value.	Complete for finite branching.
Optimality	Finds the minimum-cost path.	Finds the shallowest path when all step costs are equal.
Best Application	Mazes with different movement costs.	Large mazes with equal movement costs and unknown solution depth.

The complexity expressions describe standard search-tree formulations. For graph search with duplicate-state detection, actual memory use also depends on the number of distinct reachable states.

Relationship to Intelligent Agent Types

Different intelligent agent designs influence how the robot explores the maze.

1. Simple Reflex Agent

- Uses rules based on current sensor readings.
- Example: If the cell ahead contains a wall, turn left.
- It responds quickly but may enter loops or fail to find the exit because it does not remember previous locations.

2. Model-Based Agent

- Maintains an internal representation of discovered cells, walls, and visited positions.
- It can remember previously explored areas and avoid unnecessary revisits.
- This is especially useful when the maze is initially unknown.

3. Goal-Based Agent

- Uses the objective of reaching the exit to guide its decisions.

- It evaluates alternative routes and selects actions that help reach the goal.
- UCS or IDS can be incorporated into its planning process.

4. Utility-Based Agent

- Evaluates routes according to factors such as distance, energy consumption, time, and risk.
- It can select a safer or less costly route when multiple paths lead to the exit.
- UCS can optimize a route when the movement costs appropriately represent these factors.

Most Effective Combination

For a robot navigating a large, unknown maze, a model-based, goal-based agent using Iterative Deepening Search is a practical choice when all movements have equal cost and memory is limited.

- The model-based component stores the cells and obstacles discovered by the robot.
- The goal-based component focuses on finding the exit.
- IDS explores progressively deeper paths while keeping memory usage relatively low.
- The robot updates its internal map whenever new information becomes available.

However, if movements have different costs—for example, rough terrain consumes more battery—a model-based, goal-based agent using Uniform Cost Search is preferable because it finds a minimum-cost path under the standard UCS assumptions. In a genuinely unknown maze, the robot may not be able to construct the complete map before moving. It should therefore combine search with real-time sensing, update its map after each observation, and replan when new obstacles are discovered.

Conclusion

UCS is suitable when finding the lowest-cost path is essential, whereas IDS is suitable when the maze is unweighted, its depth is unknown, and memory is limited. Combining an appropriate search algorithm with a model-based and goal-based agent enables the robot to remember its observations, plan effectively, and reach the exit reliably.